Istanbul Urban Health Risk Prediction

Model Refinement, Test Submission & Deployment

SDG Homework 2 – Assignment 3

Data: Istanbul Metropolitan Municipality (IBB) Open Data Portal

June 2025

PART A: MODEL REFINEMENT

1. Overview

This section covers the model refinement work done for the Istanbul Urban Health Risk Prediction project. The dataset comes from the IBB Open Data Portal and combines air quality, weather, and traffic measurements across Istanbul's districts. The goal is to predict a health risk level — one of four categories — for a given observation.

The baseline model from the previous assignment performed well overall but struggled with the 'Çok Yüksek Risk (Tehlikeli)' class, which is both the rarest and the most important from a public health standpoint. The refinement phase focuses on fixing that.

Two main changes were made compared to the initial model: SMOTE was applied to handle class imbalance, and an XGBoost model was trained alongside the Random Forest to see which one performs better in practice.

2. Model Evaluation

Before refinement, the Random Forest achieved solid overall accuracy but had noticeably weaker precision for 'Çok Yüksek Risk'. The confusion matrix confirmed that most of the errors involved this class — the model sometimes confused it with 'Yüksek Risk', which is the next level down.

The correlation heatmap showed that PM2.5, maximum temperature, and traffic index were the strongest numerical predictors. District and weather condition added important context that numerical features alone couldn't capture.

2.1 Baseline metrics summary

Metric	Observation
Overall Accuracy	Above 90%
Weighted F1-Score	Above 90%
F1 – Çok Yüksek Risk	Noticeably lower than other classes
Main issue	Low precision for the minority (highest-risk) class

The learning curve showed that the model was not severely overfitting, but structural changes to the pipeline were needed to handle the imbalanced class distribution properly.

3. Refinement Techniques

Two refinement techniques were used, each targeting a different aspect of the problem.

3.1 SMOTE

SMOTE (Synthetic Minority Oversampling Technique) was applied to the training set to increase the representation of the 'Çok Yüksek Risk' class. Instead of simply duplicating existing samples, SMOTE

creates synthetic examples by interpolating between real minority-class observations. This gives the model more to learn from without memorising the same few examples repeatedly.

Importantly, SMOTE was only applied to the training set after the split — never to the test set — to avoid any data leakage into evaluation.

3.2 Class weighting

The Random Forest was also configured with a custom class weight of 5 for 'Çok Yüksek Risk'. This means getting that class wrong costs the model five times more during training, pushing it to be more careful about the highest-risk category.

3.3 XGBoost comparison

An XGBoost classifier was introduced based on feedback from the previous phase. XGBoost builds trees sequentially, with each tree correcting the errors of the previous one. It tends to generalise well on tabular data and handles class imbalance reasonably when combined with sample weights. Training both models on the same data made it easy to compare them fairly.

4. Hyperparameter Tuning

Hyperparameters were set based on domain knowledge and manual testing rather than an automated grid search. The key reasoning behind each choice is noted below.

4.1 Random Forest

Hyperparameter	Value	Why
n_estimators	300	Large ensemble for stable predictions
max_depth	None	Unconstrained depth; class weighting limits overfitting
min_samples_split / leaf	2 / 1	Standard defaults
class_weight	{vh_risk: 5, others: 1}	5x penalty for misclassifying highest-risk class
random_state	42	Reproducibility

4.2 XGBoost

Hyperparameter	Value	Why
n_estimators	300	Same as RF for a fair comparison
max_depth	12	Deeper trees to capture complex feature interactions
learning_rate	0.05	Slow, fine-grained boosting
eval_metric	mlogloss	Standard for multiclass problems
sample_weight	5 for 'Çok Yüksek Risk'	Upweights the minority class

The class weighting had the clearest impact: false negatives for 'Çok Yüksek Risk' dropped significantly after it was applied. XGBoost's deeper trees helped it pick up on non-linear relationships between district, pollution levels, and traffic that the RF handled less cleanly.

5. Cross-Validation

A 5-fold cross-validation strategy was used for generating learning curves on both models. In each fold, four-fifths of the training data was used to fit the model and the remaining fifth was used as a validation set. Scores were averaged across all five folds.

The initial train/test split used stratification (stratify=y_encoded) to ensure both sets had the same class proportions. This was kept the same throughout all refinement iterations. The only key consideration added during refinement was making sure SMOTE was applied only within training folds and never on validation data.

The learning curves showed that validation scores improved steadily with more training data and then levelled off. XGBoost had a slightly tighter gap between training and validation scores, suggesting marginally better generalisation.

6. Feature Selection

Feature selection was done before modelling, based on the correlation analysis and domain logic. The same seven features were used throughout refinement — no additional removal or transformation was applied in this phase.

Feature	Type	Rationale
İlçe (District)	Categorical → OHE	Captures spatial variation in air quality
Maks_Sıcaklik_C	Numerical	Heat-related health risk
Toplam_Yağış_mm	Numerical	Rainfall affects pollutant dispersion
Maks_Rüzgar_Hızı_kmh	Numerical	Wind disperses or concentrates pollutants
average_traffic_index	Numerical	Proxy for vehicle emissions
Ay (Month)	Numerical	Seasonal pollution patterns
Durum (Weather Condition)	Categorical → OHE	General weather state

PM2.5 turned out to be among the strongest predictors based on the correlation heatmap. The categorical variables — district and weather condition — were essential for capturing context that the raw numbers could not express on their own.

PART B: TEST SUBMISSION

1. Overview

This section describes how the trained model was applied to the held-out test set and how performance was evaluated. The test set consists of the 20% of data withheld during the initial split. It was kept completely separate throughout all refinement work to ensure an unbiased evaluation.

The same preprocessing pipeline was applied to the test set as to the training set, with one rule: any parameters used for transformations (scaling means, encoding columns) were fitted only on training data and then applied to the test set — not refitted.

2. Data Preparation for Testing

The following steps were applied to the test data in order:

- Feature selection: the same seven features were extracted from the test indices.
- One-Hot Encoding: the same column structure produced when encoding the training set was enforced on the test set — missing columns were filled with zeros, extra ones dropped.
- Label Encoding: the LabelEncoder fitted on training labels was used to encode the test targets, keeping integer-to-class mappings consistent.
- StandardScaler: the scaler fitted on the training set was used to transform the test set — not refitted.

SMOTE was not applied to the test set. It is a training-time technique and should never be used during evaluation, as it would inflate metrics by testing on synthetic data the model helped generate.

3. Model Application

Both models were applied to the scaled test features to generate predictions and probability estimates.

```
# Scale test features using the training scaler
X_test_scaled = scaler.transform(X_test)
```

```
# Hard class predictions
y_pred = model.predict(X_test_scaled)
```

```
# Probability estimates (used for Precision-Recall curves)
y_pred_prob = model.predict_proba(X_test_scaled)
```

```
# Evaluation
print(f'Accuracy: {accuracy_score(y_test, y_pred):.2%}')
print(f'F1 (weighted): {f1_score(y_test, y_pred, average="weighted"):.2%}')
print(classification_report(y_test, y_pred, target_names=class_names))
```

The same steps were repeated for the XGBoost model. All comparison visualisations — confusion matrices, error rate bars, Precision-Recall curves, and PM2.5 box plots by predicted risk — were generated from these test-set outputs.

4. Test Metrics

Both models performed well on the test set after refinement. The tables below summarise the key results.

4.1 Random Forest

Metric	Result
Overall Accuracy	> 90%
Weighted F1-Score	> 90%
Hardest class	Çok Yüksek Risk — improved but still lowest F1 of the four
Other classes	High precision and recall across Düşük, Orta, Yüksek Risk

4.2 XGBoost

Metric	Result
Overall Accuracy	Comparable to Random Forest
Weighted F1-Score	Comparable to Random Forest
Class-level error rates	Slightly lower or equal to RF across most classes

4.3 Overfitting / underfitting

The learning curves for both models showed that training scores stayed high throughout and validation scores rose steadily before levelling off. There was no sign of underfitting. The gap between training and validation accuracy was slightly wider for Random Forest, which is expected for an unconstrained ensemble. Neither model showed serious overfitting on the test set.

5. Model Deployment

For this project, deployment is conceptual rather than a live system. That said, the pipeline is built in a way that would make deployment straightforward. The model, scaler, and label encoder are all standard scikit-learn objects that can be serialised with joblib. A new observation arriving at inference time would go through the same preprocessing steps (OHE, scaling) and then be passed to the model.

In a real setting, this could be wrapped in a simple REST API using Flask or FastAPI, accepting JSON inputs and returning the predicted risk class along with the probability for each category.

6. Code Implementation

The code below covers the main steps for both the refinement and test submission phases.

6.1 Data preparation and SMOTE

```
df = pd.read_csv('Istanbul_Final_Analiz_Verisi.csv')
```

```
features = ['İlçe', 'Maks_Sıcaklik_C', 'Toplam_Yağış_mm',
            'Maks_Rüzgar_Hızı_kmh', 'average_traffic_index', 'Ay', 'Durum']
target = 'Sağlık_Risk_Durumu'
```

```
X_encoded = pd.get_dummies(X, columns=['İlçe', 'Durum'], drop_first=True)
y_encoded = LabelEncoder().fit_transform(y)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y_encoded, test_size=0.20, random_state=42, stratify=y_encoded)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

```
# SMOTE on training data only
smote = SMOTE(random_state=42)
X_train_scaled, y_train = smote.fit_resample(X_train_scaled, y_train)
```

6.2 Refined Random Forest

```
custom_weights = {i: 1 for i in range(len(class_names))}
custom_weights[vh_risk_idx] = 5  # 5x penalty for highest-risk class
```

```
model = RandomForestClassifier(
    n_estimators=300, max_depth=None,
    min_samples_split=2, min_samples_leaf=1,
    random_state=42, class_weight=custom_weights)
model.fit(X_train_scaled, y_train)
```

6.3 XGBoost

```
sample_weights = np.ones(len(y_train))
sample_weights[y_train == vh_risk_idx] = 5
```

```
xgb_model = XGBClassifier(
    n_estimators=300, max_depth=12, learning_rate=0.05,
    random_state=42, eval_metric='mlogloss')
xgb_model.fit(X_train_scaled, y_train, sample_weight=sample_weights)
```

Conclusion

The main challenge throughout this project was the imbalanced class distribution. The 'Çok Yüksek Risk (Tehlikeli)' class was the least common but the most important to get right. Applying SMOTE and a custom class weight of 5 noticeably improved the model's ability to detect high-risk cases, without hurting performance on the other classes significantly.

Comparing Random Forest and XGBoost gave a clearer picture of the trade-offs between the two approaches. Both performed similarly on overall accuracy and F1, but XGBoost showed slightly lower error rates on some classes and a narrower gap between training and validation scores.

One difficulty was tuning the class weight — going too high reduced precision by generating too many false positives for 'Çok Yüksek Risk', while going too low left recall too weak. A weight of 5 was a reasonable middle ground, though it could be refined further with more systematic hyperparameter search.

PART C: DEPLOYMENT

1. Overview

This section outlines how the trained model could be deployed for real-world use. Since this is an academic project, the deployment design is conceptual, but it follows standard industry practices closely enough that it could be implemented without major changes.

The target environment is a cloud-based setup where the model is served via a REST API. The goal is to allow authorised systems — such as public health dashboards or municipal data platforms — to send a set of district-level measurements and receive a predicted risk level in response.

2. Model Serialisation

The trained model, scaler, and label encoder are all serialised using joblib, which is well-suited for scikit-learn and XGBoost objects because it handles large numpy arrays efficiently.

Artefact	Format	Purpose
Trained model (RF / XGBoost)	joblib (.pkl)	Core prediction engine
StandardScaler	joblib (.pkl)	Feature normalisation at inference time
LabelEncoder	joblib (.pkl)	Converting integer outputs back to class names
Feature column list	JSON	Enforcing consistent input structure
Class names	JSON	Human-readable labels for the output

Artefacts would be stored in cloud object storage (e.g., S3 or Azure Blob), organised by version. Older versions are kept for rollback. Each version is tagged with the training date, dataset hash, hyperparameters, and test metrics.

3. Model Serving

The model is served using FastAPI, wrapped in a Docker container for consistent deployment across environments. In production, multiple container instances would run behind a load balancer, with autoscaling configured based on request volume.

Individual predictions are fast — a single inference on a preprocessed input takes well under 10ms. For batch jobs (e.g., a nightly district-level risk assessment), a separate async worker would process requests from a queue and write results to a database rather than handling them through the API.

4. API Integration

Endpoint	Method	Purpose
/predict	POST	Single-observation prediction

Endpoint	Method	Purpose
/predict/batch	POST	Batch prediction for multiple inputs
/health	GET	Service health check
/model/info	GET	Returns current model version and metrics

Example request — /predict

```
{ "ilce": "Kadikoy", "maks_sicaklik_c": 32.5, "toplam_yagis_mm": 0.0,
  "maks_ruzgar_hizi_kmh": 18.0, "average_traffic_index": 74.2,
  "ay": 7, "durum": "Açık" }
```

Example response

```
{ "predicted_class": "Yüksek Risk (Sağlıksız)",
  "probabilities": { "Düşük Risk": 0.04, "Orta Risk": 0.11,
                     "Yüksek Risk": 0.71, "Çok Yüksek Risk": 0.14 } }
```

5. Security Considerations

- All endpoints require a valid JWT token. Role-based access controls distinguish read-only users from admin users.
- All traffic is encrypted with TLS 1.3. Model artefacts in storage are encrypted at rest (AES-256).
- Rate limiting is enforced at the gateway to prevent abuse.
- Pydantic schemas validate and reject malformed inputs before they reach the model.
- No personally identifiable information is processed — all inputs are environmental and geographic aggregates.

6. Monitoring and Logging

Every prediction request generates a structured log entry (JSON format) recording the input features, predicted class, confidence, model version, and timestamp. Logs are sent to a centralised aggregation service for querying.

What's monitored	Alert condition
Response latency (p95)	Triggers if above 200ms
Error rate (5xx)	Triggers if above 1%
Predicted class distribution	Drift detected using KS test ($p < 0.05$)
Accuracy vs ground truth (where available)	Triggers review if drops more than 3%

Latency and error-rate alerts page the on-call team immediately. Accuracy drift and feature drift trigger a scheduled model review to decide whether retraining is needed.

PART D: PRESENTATION & SUBMISSION

5. In-Class Presentation

The presentation will walk through the full machine learning pipeline, with a focus on the decisions made during the refinement phase and the reasoning behind them. The structure will be:

- Problem statement and dataset overview — what the data is, where it comes from, and why health risk prediction matters.
- Feature selection and preprocessing decisions.
- Baseline model results and identified weaknesses — specifically the class imbalance problem.
- Refinement approach — SMOTE, class weighting, and XGBoost comparison.
- Results — side-by-side metrics, confusion matrices, and Precision-Recall curves.
- Trade-off discussion — how precision and recall were balanced for the minority class.
- High-level deployment design.

Field	Detail
Presentation Date	June 2nd
Submission Deadline	June 1st, 23:59
Format	PDF
File Name	Ibrahim Rzazade_Oğuzhan Ketenci Mehmetcan Yılmaz Zümra Büşar_
Submission	GitHub (same link as previous assignments)